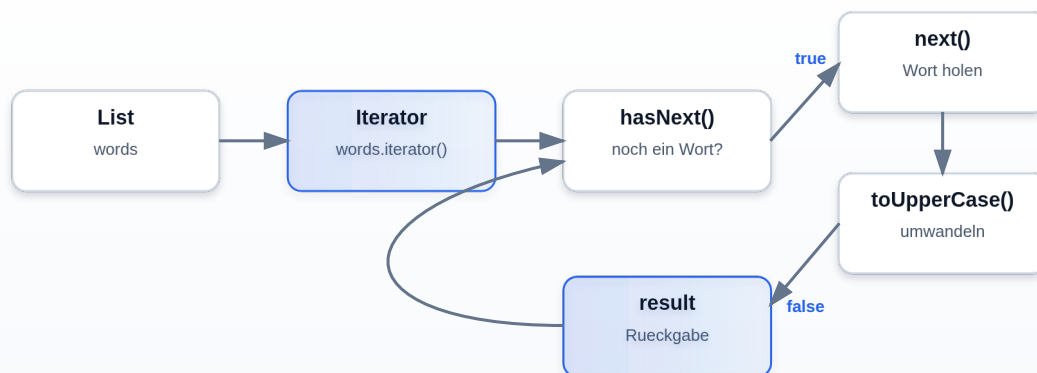


Aufgabe 01: Iterator explizit nutzen

Erklärung mit Ablaufdiagramm, zentralem Iterator-Zustand und Bezug zur konkreten Java-Implementierung.

● Professionelles Ablaufdiagramm



■ Grundidee

In dieser Aufgabe gibt es bereits eine `List<String>`. Eine Liste kann in Java einen Iterator erzeugen:

```
Iterator<String> iterator = words.iterator();
```

Der Iterator ist ein Objekt, das Schritt fuer Schritt durch die Liste geht. Der Code muss dadurch nicht wissen, wie die Liste intern aufgebaut ist.

■ Die zwei wichtigsten Methoden

Ein Iterator wird normalerweise mit zwei Methoden verwendet:

```
iterator.hasNext()
iterator.next()
```

`hasNext()` fragt:

Gibt es noch ein weiteres Element?

`next()` sagt:

Gib mir das naechste Element und gehe danach einen Schritt weiter.

Darum sieht die typische Schleife so aus:

```
while (iterator.hasNext()) {  
    String word = iterator.next();  
}
```

Die Schleife laeuft so lange, bis der Iterator kein weiteres Element mehr hat.

Die konkrete Loesung

Die Methode `collectUppercase(...)` erstellt zuerst eine leere Ergebnisliste:

```
List<String> result = new ArrayList<>();
```

Danach wird der Iterator der uebergebenen Liste erzeugt:

```
Iterator<String> iterator = words.iterator();
```

Dann wird jedes Wort gelesen, in Grossbuchstaben umgewandelt und in die Ergebnisliste eingefuegt:

```
while (iterator.hasNext()) {  
    result.add(iterator.next().toUpperCase());  
}
```

Am Ende wird die Ergebnisliste zurueckgegeben:

```
return result;
```

Ablauf am Beispiel

Eingabe:

```
List.of("java", "iterator", "pattern")
```

Der Iterator liefert nacheinander:

```
java  
iterator  
pattern
```

Die Methode wandelt jedes Wort um:

```
JAVA  
ITERATOR  
PATTERN
```

Das Ergebnis ist:

```
List.of("JAVA", "ITERATOR", "PATTERN")
```

■ Warum keine for-Schleife?

Die Aufgabe soll sichtbar machen, was hinter einer `for-each`-Schleife steckt.

Dieser Code:

```
for (String word : words) {  
    result.add(word.toUpperCase());  
}
```

verwendet intern ebenfalls einen Iterator. In dieser Aufgabe wird dieser Mechanismus nur explizit ausgeschrieben.

■ Zusammenfassung

Die Lösung besteht aus drei Schritten:

1. Iterator aus der Liste holen.
2. Mit `hasNext()` prüfen, ob noch ein Element vorhanden ist.
3. Mit `next()` das nächste Element holen und verarbeiten.

Der Iterator kapselt die Traversierung der Liste. Der eigene Code muss nur den Iterator-Vertrag verwenden.